

Exception Handling

Exception

An unexpected and unwanted event that occurs during the execution of a program and disrupts the normal flow of the program is called an Exception.

Exception = Runtime Error

Example → ZeroDivisionError

Need of Exception Handling

Exception handling is needed to:

- Prevent program crashes
- Handle errors gracefully
- Allow the program to continue running

```
x = int("abc")      # ValueError  
print("Program Continues")
```

```
try:  
    x = int("abc")  
  
except ValueError:  
    print("Invalid Input")  
  
print("Program Continues")
```

Exception Hierarchy

Exception hierarchy is the arrangement of all errors like a family tree (hierarchy).

```
BaseException
|
├── Exception
|   ├── ValueError
|   ├── TypeError
|   ├── IndexError
|   ├── KeyError
|   └── ZeroDivisionError
|
├── SystemExit
├── KeyboardInterrupt
└── GeneratorExit
```

How to Handle Exceptions

To handle exceptions we use the try-except block.

```
try:  
    # Risky code  
  
except:  
    # Code to handle error
```

```
try:  
    x = int("abc")  
  
except:  
    print("Error Occurred")
```

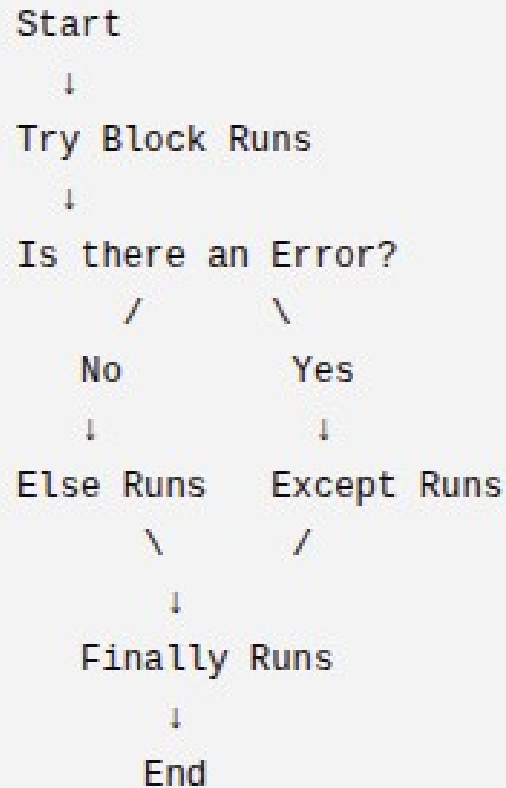
Proper Way of Handling Exceptions

Instead of catching every exception, catch the specific exception.

```
try:  
    x = int("abc")  
  
except ValueError:  
    print("Error Occurred")
```

Exception Handling Structure

```
try:  
    # Risky code  
  
except:  
    # Runs if error occurs  
  
else:  
    # Runs if no error occurs  
  
finally:  
    # Runs always
```



```
try:  
    x = 10 / 2  
  
except ZeroDivisionError:  
    print("Error")  
  
else:  
    print("Success")  
  
finally:  
    print("Done")
```

Finally Block

The finally block is the part of exception handling that always executes whether an exception occurs or not.

```
try:  
    # Risky code  
  
except:  
    # Handle error  
  
finally:  
    # Always runs
```

Uses of Finally Block

Used for cleanup work:

- Closing files
- Closing database connections
- Releasing resources

Scope Sharing

In Python, try, except, else, and finally all share the same scope.

```
try:
    f = open("file.txt")
    data = f.read()

except:
    print("Error Reading File")

finally:
    print("Closing File")
    f.close()
```

Problem with the Above Code

What if an error happens before f is created?

If the file does not exist.

UnboundLocalError

f was never assigned but finally is still trying to use it.

```
try:
    f = open("file.txt")
    data = f.read()

except:
    print("Error Reading File")

finally:
    print("Closing File")
    f.close()
```

```
f = None

try:
    f = open("file.txt")
    data = f.read()

except:
    print("Error Reading File")

finally:
    if f:
        f.close()
```

```
try:
    with open("file.txt") as f:
        data = f.read()

except:
    print("Error Reading File")
```

Advantage

- Automatically closes the file
- No need for finally

Special Case – Return and Finally

Even if a return statement is encountered, finally runs before returning.

```
def test():  
    try:  
        return 1  
  
    finally:  
        print("Finally Runs")  
  
print(test())
```

What Happens Internally?

- try block starts
- return 1 is encountered
- Python does NOT return immediately
- Return value is stored temporarily
- finally block executes
- Stored value is returned

Why?

Because finally is meant for:

- Cleanup
- Releasing resources
- Critical operations
- Python guarantees:
 - "Before leaving, I will run finally no matter what."

If finally also contains a return, then it overrides the previous return.

Without finally, the function ends immediately when return is reached.

Multiple Except Blocks

Handling different types of errors separately using multiple except blocks.

Why?

Different errors need different handling.

Example:

- Wrong input → Show one message
- Divide by zero → Show another message

```
try:
    x = int(input("Enter Number: "))
    result = 10 / x

except ValueError:
    print("Invalid Input")

except ZeroDivisionError:
    print("Cannot Divide By Zero")
```

Catch Multiple Exceptions in One Block

```
try:  
    x = int("abc")  
  
except (ValueError, TypeError):  
    print("Error Occurred")
```

Parent vs Child Exception Concept

Child Exception

- More specific exception.

Parent Exception

- More general exception.

Python stops at the first matching exception block even if a better (more specific) match exists below.

```
try:  
    x = int("abc")  
  
except Exception:  
    print("Parent Handled")  
  
except ValueError:  
    print("Child Handled")
```

```
try:  
    x = int("abc")  
  
except ValueError:  
    print("Child Handled")  
  
except Exception:  
    print("Parent Handled")
```

Using "as" in Exception Handling

Used to store the exception object in a variable so that you can access error details.

```
try:  
    x = int("abc")  
  
except ValueError as e:  
    print("Error:", e)
```

e holds the error message.

raise Keyword

Used to throw an exception manually.

```
raise MyError("message")
```

Custom Exception

A user-defined error that you create using a class to represent specific problems in your program.

Creating a Custom Exception

Step 1: Create a class inheriting from Exception

Step 2: Raise the exception

```
class MyError(Exception):  
    pass
```

```
try:  
    raise MyError("Something Went Wrong")  
  
except MyError as e:  
    print(e)
```

```
class InsufficientBalanceError(Exception):  
    pass
```

```
def withdraw(balance, amount):  
  
    if amount > balance:  
        raise InsufficientBalanceError("Not Enough Balance")  
  
    return balance - amount
```

```
try:  
    withdraw(1000, 2000)  
  
except InsufficientBalanceError as e:  
    print(e)
```

Most Important Exceptions

No.	Exception	Meaning	Example
1	ValueError	Right type, wrong value	<code>int("abc")</code>
2	TypeError	Wrong type used	<code>"2" + 2</code>
3	IndexError	Index out of range	<code>lst[5]</code>
4	KeyError	Key not found in dictionary	<code>d["b"]</code>
5	ZeroDivisionError	Division by zero	<code>10/0</code>
6	FileNotFoundError	File does not exist	<code>open("file.txt")</code>
7	AttributeError	Object has no such attribute	<code>"hello".append(1)</code>
8	NameError	Variable not defined	<code>print(x)</code>
9	ImportError	Cannot import module	<code>import unknown_module</code>
10	ModuleNotFoundError	More specific import error	<code>import abcxyz</code>

11	StopIteration	Iterator has no more values	<code>next(it)</code>
12	OverflowError	Number too large	<code>math.exp(1000)</code>
13	MemoryError	Out of memory	Huge object creation
14	RecursionError	Too much recursion	Recursive function without base case
15	AssertionError	Assertion failed	<code>assert 2 > 5</code>
16	UnboundLocalError	Local variable used before assignment	<code>print(x)</code>
17	KeyboardInterrupt	User interrupts program	<code>Ctrl + C</code>